

ArtHound MCP Server — Implementation Plan

Status: FEATURE-COMPLETE ON DEV (2026-06-17). Phases 0–3 + 5 built, applied to dev, and validated end-to-end (confirmed live in the user's Claude Code VS Code extension). Phase 4 (`submit_work_record`) remains blocked on work-actuals-tracking; scenario-planner tools are a potential later phase (§ Phase 6). Prod push deferred per user direction; working tree uncommitted. As-built reference: [docs/mcp.md](#). (Originally written 2026-06-16 to supersede a "DONE" mislabel in agent memory — the prior note was a design spec never implemented; verified then against the codebase: no `mcp` dependency, no server code, never in git history.)

Goal: expose ArtHound's canonical production data to external AI agents over the Model Context Protocol, with **scoping enforced server-side under RLS** and **mandatory actor attribution on every write**. Agents reason over PAW-shaped objects through a small, versioned tool surface — never raw SQL, never service-role.

This plan turns the [design spec](#) (held in agent memory `project_mcp_server.md`) into a sequenced build. The design principles — operations-not-queries, PAW-shaped, actor attribution, server-side scoping, read-heavy/narrow-writes — are taken as locked. What follows is *how* to land it in the current architecture (RLS live on dev+prod, four runtime identities).

0. The gating decision — agent identity & scoping

This is the security crux and **must be settled before any tool code is written**. The whole RLS migration exists so that no request reaches Postgres without a bound, scoped identity (`lib/db.py:_headers()` fails closed). An MCP agent is just another caller — it needs a real, org-scoped identity. **Service-role is off the table** (it bypasses RLS; the \$9 break-glass reachability test forbids it in the request path).

Recommended: Model A1 — agent = a service-account member, run under the existing user path

The agent authenticates to the MCP boundary with an **API key**. The server resolves the key to a dedicated **service-account auth user that is a real member of the org** (`studio_members` / `vendor_members` , `member_role = 'agent' or 'user'`), mints a short-lived Supabase-shaped JWT for that `sub` , and binds it via `set_request_token()` . From there **every existing RLS policy applies unchanged** — the agent sees exactly what that org member can see, no new policies, no new Postgres role, no ~50-table policy sweep.

- **New table `agent_credentials`** : `id` , `owner_type / owner_id` (the org), `auth_user_id` (the service-account principal), `key_hash` (argon2/bcrypt of the API key — never store the key), `label` , `scopes` (jsonb: read-only vs read+write, tool allowlist), `expires_at` , `revoked_at` , `last_used_at` . Service-role/ `arthound_system` writes only; never exposed to the agent.

- **Actor attribution** on every write: `actor_type = 'agent'`, `actor_ref = agent_credentials.id`. This makes the spec's "external agents are first-class actors" literal, not nominal.
- **Token minting**: reuse the `lib/system_auth.py` ES256 signer pattern, but mint a *user-audience* JWT (`role: authenticated`, `sub: auth_user_id`, short TTL ≤ 5 min, distinct `jti`). It is a real member token, so `resolve_my_membership` and all `*_sel` policies resolve normally.

Why A1: lowest blast radius, zero new RLS surface, reuses `get_current_user`-equivalent resolution, and the audit trail is automatic. It also means a compromised agent key is bounded by the same RLS as a compromised member — no new escalation path.

Alternative: Model A2 — dedicated `arthound_agent` Postgres role

A 5th runtime identity with `org_id` carried in JWT claims and **new per-table RLS policies** (`agent_sel` / `agent_ins` across ~50 tables). True role-level isolation, but a large migration and a parallel policy surface to keep in sync forever. **Defer** unless a customer demands Postgres-level separation between agent and human access. A1 can migrate to A2 later without changing the tool layer (only the identity-binding step changes).

Decision needed

Confirm **A1** (recommended) vs A2 before Phase 1. Everything below assumes A1.

1. Transport & runtime

Mount the MCP server as an ASGI sub-app inside the existing FastAPI process (Python `mcp` SDK, Streamable HTTP transport) at `/mcp`. One deploy, shares `db_client`, `_headers()`, `set_request_token`, env, and the connection pool. No second service to operate.

- Add `mcp` to `requirements.txt`.
- New module tree `mcp_server/` (server construction, auth middleware, tool registry) + `mcp_server/tools/` (one file per tool group: products, assets, work, estimates, schedule).
- **Per-request auth middleware** (the MCP analogue of `get_current_user`): read the agent API key from the `Authorization` header → look up `agent_credentials` by `key_hash` → check `revoked_at` / `expires_at` / scopes → mint+bind the scoped JWT → run the tool → reset the token. Mirror the request-token discipline in `lib/auth.py` exactly (bind before any DB call, reset in `finally`). Bump `last_used_at`.
- **Versioned tool names from day one**: `paw_v1.get_asset`, not `get_asset`. This is an external API contract; the cost of versioning now is trivial.
- CORS/origin: MCP is server-to-server; no browser CORS, but rate-limit per credential (ties into the open data-plane rate-limiting hardening item).

2. Tool buildability — what ships now vs what's blocked

Tool	Status	Backing data / blocker
<code>paw_v1.list_products</code>	Now	<code>replicated_products</code> (name-only until PAW product schema lands — acceptable)
<code>paw_v1.get_product</code>	Now	<code>replicated_products</code> + linked <code>replicated_assets</code> rollout
<code>paw_v1.list_assets</code>	Now	<code>replicated_assets</code> (filter by product/type/status/owner; <code>meta.__slots</code> for status/priority/team)
<code>paw_v1.get_asset</code>	Now	<code>replicated_assets</code> + product link + work summary + attachments (<code>routes/assets.py</code> helpers)
<code>paw_v1.list_work</code>	Now	<code>replicated_work</code> (+ <code>generated_work</code> for plan-side)
<code>paw_v1.get_work</code>	Now	<code>replicated_work</code> / <code>generated_work</code>
<code>paw_v1.get_estimates</code>	Now	<code>lib/estimate/</code> (<code>effective.py</code> / <code>projector.py</code>), <code>estimate_matrix</code> / <code>estimate_config</code>
<code>paw_v1.get_schedule</code>	Now	<code>lib/scheduler.py</code> (<code>build_schedule</code>)
<code>paw_v1.get_workflow_definition</code>	Now	<code>workflow_steps</code> + <code>workflow_step_dependencies</code>
<code>paw_v1.get_asset_timeline</code>	Blocked (partial v0 possible)	Rich version needs <code>paw_change_events</code> from asset-change-capture (plan locked, not built). v0 can compose <code>review_events</code> + dispatch lifecycle + <code>generated_work</code> transitions.
<code>paw_v1.flag_asset_risk</code> (write)	Now (needs small new table)	New <code>asset_flags</code> table; no dependency on heavy work
<code>paw_v1.request_human_review</code> (write)	Now (needs small new table)	New <code>review_requests</code> table, or ride <code>asset_reviews</code> with a request status
<code>paw_v1.propose_estimate_adjustment</code> (write)	Now (needs small new table)	New <code>estimate_adjustment_proposals</code> table (NumberBot pattern, externalized)
<code>paw_v1.submit_work_record</code> (write)	Blocked	No ArtHound-native work-record write target. <code>generated_work</code> is plan-side, <code>replicated_work</code> is source-owned. Needs the <code>canonical_work</code> / actuals store from work-actuals-tracking (plan only).

Cross-cutting write rule (CLAUDE.md): every write tool must resolve and attach a `canonical_asset_id`. If it can't, **abort** rather than persist an orphan. No write tool may touch a named schema slot for studio-specific data — that lives in `meta`.

3. Phases

Phase 0 — Identity & boundary (gating) — CODE COMPLETE 2026-06-16, pending dev apply

Built (model A1 confirmed): - **Migration**

`supabase/migrations/20260616000001_agent_credentials.sql` : widens the `studio_members` / `vendor_members` `member_role` CHECK to include `'agent'` (catalog-driven drop + re-add, name-independent); creates `agent_credentials` (key hash, org, principal_user_id, scopes, expiry/revoke, last_used) and append-only `agent_access_log`; both RLS-enabled + FORCE'd with a single `sys_all` policy, least-privilege grants to `arthound_system` (SELECT/UPDATE creds, SELECT/INSERT log), and `REVOKE ALL` from anon+authenticated so key hashes never reach a user token. - `lib/agent_auth.py` : `generate_key` / `hash_key` (sha256 of a 256-bit token), `resolve_credential` (under `system_identity()`, refuses unknown/revoked/expired, stamps `last_used_at`, audits), `mint_agent_token` (short-lived `authenticated` JWT via the reused `system_auth.sign_server_token` signer), `agent_identity()` context manager (bind/reset like `system_identity()`), `log_tool_event`. - `lib/system_auth.py` : extracted `sign_server_token(claims)` so system + agent tokens share one signer / one deprecation-watch path. - `scripts/agent_keys.py` : service-role admin CLI — `issue` (mint principal + `member_role='agent'` row + credential, print key once), `revoke` (revoked_at + delete membership = full off-board), `list`. - `scripts/test_agent_scoping.py` : live acceptance guard — identity, org-scoping (cross-org negative), credential secrecy, and revoke/expire lifecycle.

Guards: `scripts/rls_grant_audit.py --print-expected` already parses the two new grants; both new tables satisfy the RLS-coverage sweep (enabled+forced+policed) automatically.

Remaining to close Phase 0: apply to dev (`supabase db push`), run `rls_grant_audit.py` + `rls_persona_matrix.py` + `test_agent_scoping.py` green, then mirror to prod. No app/server code reads these tables yet, so applying is non-breaking to the running app.

Phase 1 — Read tools (the 9 "Now" reads) + transport — CODE COMPLETE + VALIDATED ON DEV 2026-06-16

Built: - `mcp_server/` package: `server.py` (FastMCP resource-server mode, `stateless_http`, mounted at `/mcp` in `main.py` with the session manager run inside the existing lifespan), `auth.py` (`AgentTokenVerifier` → `resolve_credential`, principal stashed in `AccessToken.claims`), `context.py` (`current_principal` / `authorize` / `tool_call` — scope enforcement + `agent_identity` binding + audit), `shaping.py` (slot reads + `_META_HIDDEN` cleaner). - **9 tools** (`mcp_server/tools/`), versioned `paw_v1_*`, PAW-shaped (`get_asset` pre-joins product + work rollup): `list_products`, `get_product`, `list_assets`, `get_asset`, `list_work`, `get_work`, `get_estimates` (reuses

`lib/estimate/effective.resolve_effective_matrix`), `get_schedule` (reuses `lib/scheduler.build_schedule`, studio-only), `get_workflow_definition`. - `mcp ≥ 1.28.0` added to `requirements.txt`; `lib/system_auth.sign_server_token` reused by the agent token mint.

Validated on dev via the MCP SDK client: `initialize` + `list_tools` (9), tool calls return org-scoped PAW JSON, invalid key rejected. Smoke-test credentials cleaned up afterward.

Notes / deferred: - Tools returned plain dicts → text-only at first; **resolved in Phase 5** by annotating returns as `dict[str, Any]`, so FastMCP now also emits `structuredContent`. - Streaming for `get_schedule/timeline` deferred (payloads are small in practice). - **Unrelated pre-existing bug observed:** `GET /health` 503s under `USE_USER_IDENTITY=1` (it does an identity-bound DB read but is unauthenticated/no bound token, and there is no global token-binding middleware). One-line fix = wrap its read in `system_identity()`. Out of scope for MCP; flagged.

Phase 2 — Lightweight writes (3 proposal/flag tools) — BACKEND COMPLETE + VALIDATED ON DEV 2026-06-16

Built + applied to dev (migration `20260616000002_mcp_agent_writes.sql`): - Tables `asset_flags`, `review_requests` (both `canonical_asset_id` NOT NULL), and `estimate_adjustment_proposals` (`canonical_asset_id` NULLABLE by design — estimate is a tier above any asset, the documented `estimate_share` carve-out; `workflow_step_id` is the subject). All RLS enabled + FORCE'd, `authenticated` policies via `is_my_org(owner_type, owner_id)`, `arthound_system` `SELECT` + `sys_read` for future notification consumption. `actor_type` / `actor_ref` on every row, not FK'd (audit outlives the credential). - Tools `mcp_server/tools/writes.py`: `paw_v1_flag_asset_risk`, `paw_v1_request_human_review`, `paw_v1_propose_estimate_adjustment` — all `write=True` (require a write-scoped credential), `resolve` + attach the canonical asset (abort on orphan), stamp actor attribution. 12 tools total registered.

Validated on dev via MCP SDK client: write-scoped key writes (row lands owner-scoped + agent-attributed), read-scoped key is denied ("read-only"), orphan write refused, tier-above-asset proposal works. Persona matrix 16/0 (no regression); grant-audit parses the 3 new grants. Test artifacts cleaned up.

Phase 2 UI — COMPLETE + VALIDATED ON DEV 2026-06-16 (placement: per-asset + home widget): - Backend read/triage API `routes/agent_activity.py` (mounted `/api/agent-activity`): `GET /recent` (org roll-up, asset names resolved), `GET /asset/{canonical_asset_id}`, `PATCH /flags|review-requests|proposals/{id}` (status triage, validated). Runs under the caller's RLS identity. - Frontend: `AgentActivityList` (shared renderer — kind/severity Pills, StatusDot via `statusColors`, triage Select), `AgentTab` (Asset viewer tab, per-asset + inline status triage), `AgentActivityWidget` (StudioHome + VendorHome roll-up). Build passes; lint matches the codebase's existing tab idiom. - Validated e2e under a real user JWT: `recent/asset` reads 200, `PATCH`→resolved 200, invalid status 400. Dev left clean (0 test artifacts).

Phase 3 — Timeline v0 — CODE COMPLETE + VALIDATED ON DEV 2026-06-16

- `mcp_server/tools/timeline.py` → `paw_v1_get_asset_timeline` (13th tool): composes `payload_dispatches` lifecycle (sent/revoked — `received_at` was dropped in 202605030000004, ingest receipt is vendor-scoped `payload_field_mappings.ingested_at`, out of v0) + `review_events` (via `review_assets`) into a newest-first, actor-attributed stream. Returns `partial: true` + a `coverage` note so a consumer knows absence ≠ no-change. Sub-query failures raise (no silent empty timeline — this caught the `received_at` bug during the smoke test).
- **Validated on dev:** an asset with dispatch history returned 21 events (11 sent + 10 revoked), org-scoped under the agent identity.
- **v1 (later):** read `paw_change_events` via `lib/changes.py` once `asset-change-capture` lands — adds source-field history (status/estimate changes) the v0 cannot see.

Phase 4 — `submit_work_record` (depends on work-actuals)

- Blocked until `work-actuals-tracking` lands a `canonical_work` / ArtHound-native work store to write into. Until then there is no correct write target (writing to `replicated_work` would corrupt source-owned replica state — forbidden by the canonical-layer principle).
- When unblocked: `submit_work_record` writes an ArtHound-owned work record with mandatory actor attribution, parent-work provenance, asset + workflow-step links, and outputs.
- **Acceptance:** record is canonical-asset-linked, provenance-chained, actor-attributed; no source-tool mutation.

Phase 5 — Hardening & docs — COMPLETE + VALIDATED ON DEV 2026-06-17

- **Structured output:** tool return annotations are `dict[str, Any]`, so FastMCP emits a generic object `outputSchema` and `structuredContent` on every call (verified live — `count` read back via `structuredContent`), alongside the text block. All 13 tools carry a schema.
- **Per-credential rate limiting:** `mcp_server/ratelimit.py` token bucket keyed on credential id, enforced in `mcp_server/context.py:tool_call` (denied → audit `denied / rate_limited` + `ToolError`). Tunable via `MCP_RATE_LIMIT_PER_MIN` (default 120) / `MCP_RATE_LIMIT_BURST` (default 30); `0` disables. Per-worker/in-process — shared (Redis) limiter is the future upgrade. Logic unit-verified (burst=2 → allow,allow,deny,deny,deny; per-credential isolation holds).
- **last_used_at** + **access log:** already shipped in Phase 0 (`agent_access_log`).
- **Docs:** `mcp_server/README.md` (connect, identity/scoping, key mgmt, rate limits, errors, full tool catalog, versioning policy `paw_v1→paw_v2`) + a CLAUDE.md "MCP Server" section pointing at it.

MCP server is now feature-complete on dev (Phases 0–3 + 5; Phase 2 UI done). Only Phase 4 `submit_work_record` remains, blocked on work-actuals-tracking. Prod push deferred per user direction.

4. Deliberately absent (carried from the spec)

- No `update_asset_status` / `modify_schedule` / `delete_work` — status changes flow through `submit_work_record`; schedule derives from estimates + dependencies; work is superseded, not deleted.
 - No `raw_query` / SQL surface — it throws away PAW semantics and the scoping guarantees.
 - No `bulk_*` — deferred until a concrete use case has a clear safety story (bulk is where agents do the most damage with the least visibility).
 - **No scenario-planner tools** — not exposed today. The `scenario_*` tables are studio-scoped and would slot into the same RLS model, so read tools (`paw_v1_list_scenarios` / `paw_v1_get_scenario`) are a straightforward future **Phase 6**; triggering generation (cost + async gen loop) is a heavier design decision deferred with it.
-

Phase 6 — Scenario tools (potential, not built)

Read tools over the scenario planner's session-scoped output (a generated scenario's products/assets/work, horizon, counts). Studio-only; no-op for vendor keys. Reads only — the `scenario_*` tables carry `studio_id`, so they scope under the agent's existing RLS with no new policy surface. Triggering generation (`run_scenario`) is a separate, heavier decision (AI token cost, long-running gen loop, a different trust model than reading) and would want its own write-scope + cost/rate controls.

Decisions (settled during the build)

1. **Identity model:** A1 (service-account member under existing RLS). A2 (dedicated `arthound_agent` Postgres role) deferred — A1 can upgrade to it later without touching the tool layer.
2. **Runtime:** mounted ASGI sub-app at `/mcp` inside the FastAPI process.
3. **request_human_review storage:** new `review_requests` table (keeps the immutable review lifecycle clean).
4. **Key issuance UX:** CLI (`scripts/agent_keys.py`); a Settings UI can come later.
5. **Scope granularity:** per-tool allowlist + read/write mode from day one (`agent_credentials.scopes`).